

Machine Learning for social sciences

Session 0: Git & Virtual Environments

William Aboucaya

Today's Roadmap

- ➊ Introduction: why this matters
- ➋ Basics of command-line interfaces
- ➌ Git & version control
- ➍ Dependency management with `uv`
- ➎ Putting it all together: an end-to-end demo

This session is hands-on: keep a terminal open and follow along on slides marked **LIVE DEMO**.

Section: Introduction

- 1 Introduction
- 2 Command-Line Basics
- 3 Git & Version Control
- 4 Dependency Management with uv
- 5 Putting It All Together

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.
- You emailed yourself `project_final_v3_REAL_final.py`.

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.
- You emailed yourself `project_final_v3_REAL_final.py`.
- Your code runs perfectly on your laptop ... and crashes the moment a classmate tries it.

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.
- You emailed yourself `project_final_v3_REAL_final.py`.
- Your code runs perfectly on your laptop ... and crashes the moment a classmate tries it.
- Two people edited the same notebook and now someone's changes are gone.

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.
- You emailed yourself `project_final_v3_REAL_final.py`.
- Your code runs perfectly on your laptop ... and crashes the moment a classmate tries it.
- Two people edited the same notebook and now someone's changes are gone.
- A library update silently broke a project you hadn't touched in months.

A Few Familiar Situations...

- You broke your working code and have no way back to the version that worked yesterday.
- You emailed yourself `project_final_v3_REAL_final.py`.
- Your code runs perfectly on your laptop ... and crashes the moment a classmate tries it.
- Two people edited the same notebook and now someone's changes are gone.
- A library update silently broke a project you hadn't touched in months.

All of these have a well-established fix, and that's what this lesson is about.

Two Separate Problems, Two Tools

Tracking changes to *code*

- Who changed what, and when?
- Can I go back to an earlier version?
- Can two people work on the same project without overwriting each other?

→ **Git**

Tracking *dependencies*

- Which Python & package versions does this project need?
- Can someone else reproduce my setup?
- Can I keep two projects' dependencies from colliding?

→ **uv**

Two Separate Problems, Two Tools

Tracking changes to *code*

- Who changed what, and when?
- Can I go back to an earlier version?
- Can two people work on the same project without overwriting each other?

→ **Git**

Tracking *dependencies*

- Which Python & package versions does this project need?
- Can someone else reproduce my setup?
- Can I keep two projects' dependencies from colliding?

→ **uv**

Together, they answer: *“can someone else run my project, exactly as I intended, six months from now?”*

Section: Command-Line Basics

- 1 Introduction
- 2 Command-Line Basics**
- 3 Git & Version Control
- 4 Dependency Management with uv
- 5 Putting It All Together

Why the Terminal?

- Everything we do today (Git and uv) happens by typing commands in a **terminal**, not by clicking around.
- A terminal is just a text-based way to talk to your computer: instead of double-clicking folders, you *type* where you want to go and what you want to do.
- This section is a quick, practical primer: just enough to be comfortable for the rest of today.

Why the Terminal?

- Everything we do today (Git and uv) happens by typing commands in a **terminal**, not by clicking around.
- A terminal is just a text-based way to talk to your computer: instead of double-clicking folders, you *type* where you want to go and what you want to do.
- This section is a quick, practical primer: just enough to be comfortable for the rest of today.

Where to open one

Linux : your distribution's terminal

Windows : “Git Bash” (installed alongside Git) is recommended for today, so commands match everyone else's

macOS : the “Terminal” app

The Prompt

- When you open a terminal, you'll see a **prompt**. It's waiting for you to type a command and press Enter
- In these slides, every line starting with \$ is a command *you* type (the \$ itself is not part of what you type)
- Lines without a \$ are the computer's **output** → what it prints back

```
$ echo "this is a command"  
this is a command
```

Where Am I? - pwd and ls

LIVE DEMO

```
$ pwd
/home/William/Documents

$ ls
Desktop  Documents  Downloads  Pictures
```

- pwd (“print working directory”): where am I right now?
- ls (“list”): what’s in this folder?


```
$ cd Documents          # move into a folder
$ pwd
/home/William/Documents

$ cd ..                # move up one level
$ pwd
/home/William

$ cd Documents/my-project # move several levels at once
```

- cd (“change directory”) is how you navigate. This is the terminal equivalent of double-clicking a folder.
- cd .. always means “go up one level”, no matter where you are.

```
$ mkdir my-project          # create a new folder
$ cd my-project
$ touch main.py             # create a new, empty file
$ ls
main.py
```

- `mkdir` (“make directory”): create a folder.
- `touch`: create an empty file.

```
$ clear                # clear the screen (visual only)
$ rm old_file.py       # delete a file - no trash bin!
$ rm -r old_folder/    # delete a folder + its contents
```

Careful with `rm`

Unlike deleting a file in a file explorer, `rm` does **not** send it to a Recycle Bin/Trash: it's gone immediately. Double-check what you're deleting.

- Tip: press the **up arrow** to bring back your previous command instead of retyping it.

Command-Line Basics Recap

Command	Objective
<code>pwd</code>	Where am I?
<code>ls</code>	What's here?
<code>cd <folder> / cd ..</code>	Move into / up out of a folder
<code>mkdir <name></code>	Create a folder
<code>touch <name></code>	Create an empty file
<code>rm / rm -r</code>	Delete a file / a folder (careful!)
<code>clear</code>	Clear the screen

Command-Line Basics Recap

Command	Objective
<code>pwd</code>	Where am I?
<code>ls</code>	What's here?
<code>cd <folder> / cd ..</code>	Move into / up out of a folder
<code>mkdir <name></code>	Create a folder
<code>touch <name></code>	Create an empty file
<code>rm / rm -r</code>	Delete a file / a folder (careful!)
<code>clear</code>	Clear the screen

That's genuinely enough to follow along with everything ahead: every Git and uv command builds on exactly this.

Section: Git & Version Control

- 1 Introduction
- 2 Command-Line Basics
- 3 Git & Version Control**
- 4 Dependency Management with uv
- 5 Putting It All Together

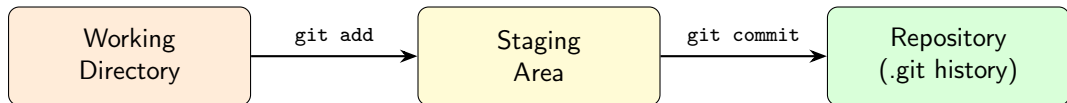
What Is Git?

- **Git** is a *version control system*: it records the history of a project over time.
- Instead of one folder that keeps changing, you get a **timeline of snapshots** you can inspect, compare, or return to.
- It runs entirely **locally** on your machine, no internet connection required to use it.
- Created by Linus Torvalds (2005) to manage the Linux kernel's source code; now the de facto standard everywhere.

Key idea

A Git repository is not just your current files: it's your current files *plus their entire history*.

The Three Areas of a Git Project



- **Working directory:** the files you're currently editing.
- **Staging area:** a holding zone: "these are the changes I'm about to save."
- **Repository:** the permanent, versioned history of committed snapshots.

- Git is usually pre-installed on macOS/Linux; on Windows, install *Git for Windows*.
- Before your first commit, tell Git who you are: this gets attached to every commit you make.

```
$ git --version
$ git config --global user.name "William Aboucaya"
$ git config --global user.email "william@example.fr"
```

```
$ mkdir my-project && cd my-project  
$ git init  
Initialized empty Git repository in ../my-project/.git/
```

- `git init` creates a hidden `.git/` folder – that's the entire repository.
- Delete `.git/` and the folder is just... a folder again. No magic, no external service involved.

```
$ echo "print('hello')" > main.py
$ git status
On branch main
No commits yet
Untracked files:
  (use "git add <file>..." to include in what will be committed)
    main.py
```

- `git status` is your compass → run it constantly. It tells you exactly what Git sees as changed, staged, or untracked.

```
$ git add main.py
$ git status
Changes to be committed:
  new file:   main.py

$ git commit -m "Add initial hello world script"
[main (root-commit) a1b2c3d] Add initial hello world script
1 file changed, 1 insertion(+)
```

- `git add` stages a file (or `git add .` to stage everything).
- `git commit -m "..."` permanently records a snapshot, with a message describing *why*.

A Commit Message That Helps Future You

Avoid

- "fix"
- "update"
- "asdf"
- "final version 2"

Prefer

- "Fix off-by-one error in loop"
- "Add CSV export for results"
- "Remove unused import"

A good rule: *"if I apply this commit, it will ..."* → the message should complete that sentence.

```
$ git log
commit a1b2c3d4e5f6...
Author: William Aboucaya <william@example.fr>
Date:   Tue Jul 21 10:03:00 2026
```

```
    Add initial hello world script
```

```
$ git log --oneline
a1b2c3d Add initial hello world script
```

- Every commit has a unique ID (hash), an author, a date, and a message.
- `-oneline` gives a compact view, handy once history grows.

```
$ echo "print('goodbye ')" >> main.py
$ git diff
--- a/main.py
+++ b/main.py
@@ -1,2 @@
    print('hello')
+print('goodbye ')
```

- git diff shows exactly what changed since the last commit, line by line.
- Lines starting with + are additions, - are removals.

Ignoring Files: .gitignore

LIVE DEMO

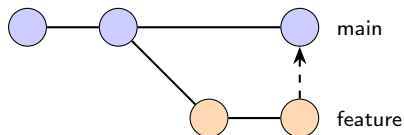
Not everything belongs in version control: generated files, caches, secrets, virtual environments. . .

```
$ cat .gitignore
__pycache__/
*.pyc
.venv/
.env
.ipynb_checkpoints/
```

- Any file/folder matching a pattern in .gitignore is skipped by `git status` and `git add`.
- Create it *early*: once a file is already tracked, ignoring it later takes an extra step.

Why Branch?

- A **branch** is an independent line of work: you can experiment without touching the stable version.
- Typical use: `main` always stays working; new features are built on their own branch, then merged in once ready.



```
$ git branch                # list branches
* main
$ git switch -c feature-greeting # create + switch in one step
Switched to a new branch 'feature-greeting'
$ git branch
* feature-greeting
  main
```

- `git switch -c <name>` is the modern way to create and move to a new branch (older tutorials use `git checkout -b`).

```
$ echo "print('feature added')" >> main.py
$ git add main.py
$ git commit -m "Add feature greeting"
$ git switch main
$ git merge feature-greeting
Updating a1b2c3d..f4e5d6c
Fast-forward
 main.py | 1 +
```

- Switch back to the branch you want to merge *into* (usually main), then run `git merge <branch>`.

When Merges Collide: Conflicts

[LIVE DEMO](#)

Conflicts happen when the *same lines* were changed differently on both branches.

```
$ git merge feature-x
Auto-merging main.py
CONFLICT (content): Merge conflict in main.py
Automatic merge failed; fix conflicts and then commit.
```

Git marks the conflicting section directly in the file:

```
<<<<<< HEAD
print('hello from main')
=====
print('hello from feature-x')
>>>>>> feature-x
```

- 1 Open the file, decide what the final code should look like.
- 2 Remove the <<<<<<, =====, >>>>>> markers entirely.
- 3 Stage and commit the resolution.

```
# after manually editing main.py:  
$ git add main.py  
$ git commit -m "Resolve conflict: keep both greetings"
```

Reassurance

Conflicts are normal, not a sign something went wrong. They just mean Git needs a human decision.

What Is GitHub?

- **Git** is the tool that tracks history *on your machine*. **GitHub** is a separate, online service that *hosts* Git repositories.
- GitHub adds: a backup off your laptop, a way to share/collaborate, pull requests, issue tracking, project pages.
- Git works perfectly well with **no** GitHub account at all. GitHub is one popular place to put a *remote* copy of your repository.

Alternatives to GitHub

GitLab, Bitbucket, and self-hosted options (Gitea, self-hosted GitLab) all serve the same purpose: hosting Git repositories with extra collaboration features.

```
$ git remote add origin https://github.com/WilliamAboucaya/my-  
  project.git  
$ git remote -v  
origin  https://github.com/WilliamAboucaya/my-project.git (fetch)  
origin  https://github.com/WilliamAboucaya/my-project.git (push)  
$ git push -u origin main
```

- origin is just a conventional nickname for "my main remote."
- -u remembers this link, so future pushes can just be git push.

```
# on a different machine, or a teammate's machine:  
$ git clone https://github.com/WilliamAboucaya/my-project.git  
$ cd my-project  
  
# after someone else has pushed new commits:  
$ git pull
```

- `git clone` downloads a full copy of a remote repository, history included.
- `git pull` fetches and merges new commits from the remote into your local branch.

Collaborating: Pull Requests

- A **pull request** (PR) proposes merging one branch into another. This is commonly used even for solo projects, to review your own change before merging.
- Typical flow: push a feature branch to GitHub → open a PR → teammates comment/review → merge once approved.
- This is a *GitHub* feature (not Git itself). GitLab/Bitbucket have equivalents under different names.

- Git tracks history locally: **working directory** → **staging** → **repository**
- Core loop: `status` → `add` → `commit`, with `log/diff` to inspect history
- `.gitignore` keeps generated/sensitive files out of version control
- **Branches** isolate work; **merging** brings it back together (conflicts are normal, not errors)
- **GitHub** $\neq$ Git: it's a hosting/collaboration layer on top

Section: Dependency Management with uv

- 1 Introduction
- 2 Command-Line Basics
- 3 Git & Version Control
- 4 Dependency Management with uv**
- 5 Putting It All Together

Why Isolate Dependencies?

- Every project depends on specific versions of Python packages (and sometimes Python itself).
- Install everything **globally** on your machine, and sooner or later:
 - Project A needs pandas 1.5, Project B needs pandas 2.2 → only one can win globally.
 - You upgrade a package for one project and silently break another.
 - A teammate (or your future self) can't tell *which* versions your project actually needs.

Key idea

Each project should have its own isolated, self-contained set of dependencies: an **environment**.

The Traditional Approach: venv + pip

- venv: built into Python, creates an isolated folder with its own Python interpreter and packages.
- pip: installs packages into whichever environment is currently active.

```
$ python -m venv .venv
$ source .venv/bin/activate      # (or .venv\Scripts\activate on
    Windows)
(.venv) $ pip install pandas
(.venv) $ pip freeze > requirements.txt
```

- This is the classic, everywhere-supported workflow, you'll see it in most existing Python projects and tutorials.

Why We Won't Use `venv` + `pip` Today

- **Manual activation:** easy to forget, easy to silently install into the wrong environment.
- **No real lockfile:** `requirements.txt` records *what you asked for*, not always the exact resolved versions everyone will get.
- **Managing Python versions is a separate problem:** `venv` assumes you already have the right Python installed.
- **Slow:** resolving and installing packages with `pip` can take a while on larger projects.

Why We Won't Use `venv` + `pip` Today

- **Manual activation:** easy to forget, easy to silently install into the wrong environment.
- **No real lockfile:** `requirements.txt` records *what you asked for*, not always the exact resolved versions everyone will get.
- **Managing Python versions is a separate problem:** `venv` assumes you already have the right Python installed.
- **Slow:** resolving and installing packages with `pip` can take a while on larger projects.

⇒ None of these are fatal, they're just friction. Today, we'll use a tool that removes most of it.

- uv is a modern, all-in-one Python project manager: **Python versions + virtual environments + packages + lockfile**, one tool, one command family.
- Built in Rust: dramatically faster installs than pip.
- No manual "activate" step required for day-to-day use (uv run handles it for you).
- Produces a real **lockfile**, so "it works on my machine" becomes "it works on every machine."

Where it fits

Everything we're about to do replaces the venv + pip + requirements.txt workflow from the previous slides → same goal, smoother path.


```
# macOS / Linux
$ curl -LsSf https://astral.sh/uv/install.sh | sh

# Windows (PowerShell)
$ powershell -c "irm https://astral.sh/uv/install.ps1 | iex"

$ uv --version
uv 0.12.1
```

- One-time setup: after this, uv is available in any terminal, for any project.

```
$ uv init my-analysis
$ cd my-analysis
$ ls
main.py  pyproject.toml  README.md  .python-version
```

- `uv init` scaffolds a new project, including a `pyproject.toml`, the standard place Python projects declare their dependencies.
- `.python-version` pins exactly which Python version this project uses. It can be set using `uv init -p 3.x` to ask for the project to rely on Python version 3.x
- Other files are not necessarily relevant, their generation can be disabled by running `uv init --bare my-analysis` instead

```
$ uv add pandas numpy
Resolved 12 packages in 320ms
Installed 12 packages in 180ms
+ numpy==2.2.1
+ pandas==2.2.3
...
```

- `uv add <package>` installs a package **and** records it in `pyproject.toml` → no separate “remember to freeze” step.
- Behind the scenes, `uv` creates a `.venv/` folder automatically: you never had to type `venv` yourself.

```
$ uv run main.py  
Hello from my-analysis!
```

```
$ uv run python  
>>> import pandas as pd  
>>> pd.__version__  
'2.2.3'
```

- `uv run` executes a command *inside* the project's environment automatically, no source `.venv/bin/activate` needed.
- Works for scripts, an interactive interpreter, Jupyter, pytest, anything.

```
$ cat pyproject.toml
[project]
name = "my-analysis"
requires-python = ">=3.12"
dependencies = [
    "numpy>=2.2.1",
    "pandas>=2.2.3",
]
```

- `pyproject.toml`: the dependencies you *asked for* (human-editable, high-level).
- `uv.lock`: the *exact* resolved versions of everything, including sub-dependencies (machine-generated, never edit by hand).

```
# on a teammate's machine, after cloning the repo:
$ git clone https://github.com/WilliamAboucaya/my-analysis.git
$ cd my-analysis
$ uv sync
Resolved 12 packages in 4ms
Installed 12 packages in 95ms
```

- `uv sync` reads `uv.lock` and installs *exactly* those versions → same environment, guaranteed.
- This is the payoff: the lockfile is what makes “works on my machine” obsolete.

Managing Python Versions

uv can also install and pin Python itself → no separate installer needed.

```
$ uv python install 3.12
$ uv python pin 3.12
Pinned .python-version to 3.12
```

⇒ Remember installing Python at the very start of this course? uv can handle that step too, per-project, going forward.

```
$ cat .gitignore
.venv/
__pycache__/
*.pyc

$ git add pyproject.toml uv.lock .gitignore main.py
$ git commit -m "Set up project with uv"
```

- **Ignore** `.venv/`: it's regenerable, and shouldn't live in version control.
- **Commit** `pyproject.toml` *and* `uv.lock`: together, they let anyone reproduce your exact environment with one command.

Part 2 Recap

- Isolated environments prevent dependency conflicts between projects
- `uv` replaces `venv` + `pip` + `requirements.txt` with one faster, simpler tool
- Core loop: `uv init` → `uv add` → `uv run`
- `pyproject.toml` (what you want) + `uv.lock` (exactly what you get) → `uv sync` to reproduce
- `uv` can also install/pin Python versions themselves
- Commit `pyproject.toml` & `uv.lock`; ignore `.venv/`

Part 2 Recap

- Isolated environments prevent dependency conflicts between projects
- uv replaces venv + pip + requirements.txt with one faster, simpler tool
- Core loop: `uv init` → `uv add` → `uv run`
- `pyproject.toml` (what you want) + `uv.lock` (exactly what you get) → `uv sync` to reproduce
- uv can also install/pin Python versions themselves
- Commit `pyproject.toml` & `uv.lock`; ignore `.venv/`

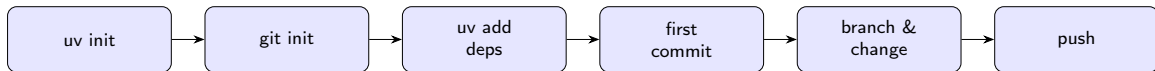
Next: putting Git and uv together, end to end

Section: Putting It All Together

- 1 Introduction
- 2 Command-Line Basics
- 3 Git & Version Control
- 4 Dependency Management with uv
- 5 Putting It All Together

The Goal

- We've seen Git and uv as two separate topics. In practice, you use them **together, every day**, on the same project.
- Let's build one small project from nothing, end to end, using both.



Step 1: Start the Project

LIVE DEMO

```
$ uv init weather-stats
$ cd weather-stats
$ ls
main.py  pyproject.toml  README.md  .python-version
```

- `uv init` gives us a working Python project skeleton before Git even enters the picture.

Step 2: Bring in Git

LIVE DEMO

```
$ git init
$ echo ".venv/" > .gitignore
$ git status
Untracked files:
  .gitignore
  .python-version
  README.md
  main.py
  pyproject.toml
```

- Notice `.venv/` doesn't even exist yet: we're ignoring it *before* it's created, which is the right order.

Step 3: Add Dependencies

LIVE DEMO

```
$ uv add pandas
Resolved 8 packages in 210ms
Installed 8 packages in 140ms
+ pandas==2.2.3
...

$ git status
Untracked files:
  .gitignore  .python-version  README.md
  main.py    pyproject.toml  uv.lock
```

- `uv.lock` appeared automatically and Git sees it as just another file to track.

Step 4: First Commit

LIVE DEMO

```
$ git add .  
$ git status  
Changes to be committed:  
  new file:   .gitignore  
  new file:   README.md  
  new file:   main.py  
  new file:   pyproject.toml  
  new file:   uv.lock  
  
$ git commit -m "Initial project setup with uv"
```

- `.venv/` is absent from this list, exactly as intended.

Step 5: Work on a Feature Branch

LIVE DEMO

```
$ git switch -c add-average-temp

# edit main.py to add a function, then try it:
$ uv run python -c "from main import average_temp; \
print(average_temp([12, 15, 9]))"
12.0
```

- Same habits as before: branch first, then work (`uv run` means the right environment is always used, without thinking about activation)

Step 6: Commit and Push

LIVE DEMO

```
$ git add main.py
$ git commit -m "Add average_temp function"
$ git switch main
$ git merge add-average-temp
$ git remote add origin https://github.com/WilliamAboucaya/weather
  -stats.git
$ git push -u origin main
```

- The project (code, dependency declarations, *and* lockfile) is now backed up and shareable.

Step 7: A Teammate Picks It Up

LIVE DEMO

```
# on a completely different machine:
$ git clone https://github.com/WilliamAboucaya/weather-stats.git
$ cd weather-stats
$ uv sync
Resolved 8 packages in 5ms
Installed 8 packages in 90ms

$ uv run python -c "from main import average_temp; \
print(average_temp([12, 15, 9]))"
12.0
```

- Two commands (`git clone`, `uv sync`) and a teammate has the *exact* same project, same dependency versions, ready to run.

Why All This Matters

Without Git + uv

"Which version of this file is the good one?"

"It works on my machine"

Manual zip files / email attachments

Overwriting a teammate's work

With Git + uv

Full history, always recoverable

Locked, reproducible dependencies

`git clone`

Branches & merges

Command	Objective
<code>git init</code>	Start a repository
<code>git status</code>	What's changed?
<code>git add <file></code>	Stage a change
<code>git commit -m "..."</code>	Save a snapshot
<code>git log / git diff</code>	Inspect history
<code>git switch -c <name></code>	create & Move to a branch
<code>git merge <branch></code>	Bring a branch back in
<code>git clone <url></code>	Copy a remote repository
<code>git pull / git push</code>	Sync with a remote

Command	Objective
<code>uv init <name></code>	Start a new project
<code>uv add <package></code>	Add & record a dependency
<code>uv run <cmd></code>	Run something in the project's env
<code>uv sync</code>	Install exactly what's in <code>uv.lock</code>
<code>uv python install/pin</code>	Manage Python versions

Reminder

Don't forget to commit `pyproject.toml` and `uv.lock`, and to `.gitignore .venv/`.

Recap: The Full Picture

- **Git** gives every project a history you can inspect, branch, and safely collaborate on.
- **GitHub** (or similar) hosts that history remotely, so it's backed up and shareable.
- **uv** gives every project its own reproducible, isolated set of dependencies, including Python itself.
- Together: `git clone + uv sync` is basically a guarantee that a project runs the same way everywhere.

Recap: The Full Picture

- **Git** gives every project a history you can inspect, branch, and safely collaborate on.
- **GitHub** (or similar) hosts that history remotely, so it's backed up and shareable.
- **uv** gives every project its own reproducible, isolated set of dependencies, including Python itself.
- Together: `git clone + uv sync` is basically a guarantee that a project runs the same way everywhere.

These habits, established now, will save you a lot of time in the data science & ML course ahead, especially for your projects.

Questions?

Disclaimer

Today's repo/demo files are available for you to revisit and practice with at <https://github.com/WilliamAboucaya/MLClass-M2IREN>